CS336
Recitation 2 Sep 22

Made by: Zihan Chen

How to use iLab

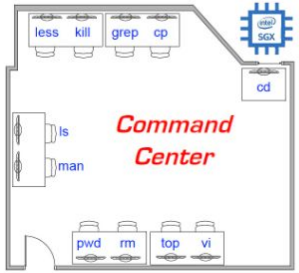
- Open your account
  - https://services.cs.rutgers.edu/accounts/
  - Take references from iLab_link.txt @ canvas/Files/Info/iLabs
    - Account start page: https://resources.cs.rutgers.edu/docs/new-users/getting-started/
    - Live list of iLab machines and current load
    - Use Meltdown, Command and CAVE machine clusters
- Connect to iLab machines:
  - Windows: Windows Terminal
    - https://apps.microsoft.com/store/detail/windows-terminal/9N0DX20HK701
  - Linux & Mac: Terminal
- The file system on iLab machines are shared, you can login to any iLab machine

Available iLab machine list

Status of ilab:

https://report.cs.rutgers.edu/nagiosnotes/iLab-machines.html

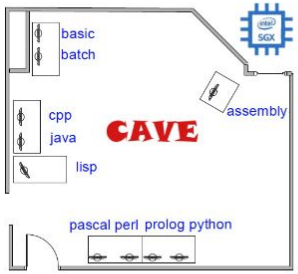
▶ Specifications



H248 Machines Status (11 units)

Hostname	GPULoad	Logins	User_Cpu	System_Cpu	Misc
cd.cs.rutgers.edu	0 %	1	1.43	0.25	
cp.cs.rutgers.edu	0 %	4	0.25	0.08	
grep.cs.rutgers.edu	0 %	1	0.60	4.65	
kill.cs.rutgers.edu	0 %	1	0.20	0.08	
less.cs.rutgers.edu	0 %	1	0.48	0.33	
ls.cs.rutgers.edu	0 %	0	0.33	0.03	
man.cs.rutgers.edu	0 %	1	0.40	0.23	
pwd.cs.rutgers.edu	0 %	1	0.30	0.10	
rm.cs.rutgers.edu	0 %	0	0.32	0.07	
top.cs.rutgers.edu	0 %	1	0.25	0.15	
vi.cs.rutgers.edu	0 %	0	0.65	0.10	

▶ Specifications.



H252 Machines Status (10 units)

Hostname	GPULoad	Logins	User_Cpu	System_Cpu	Misc
assembly.cs.rutgers.edu	0 %	0	0.62	0.12	
basic.cs.rutgers.edu	0 %	0	0.47	0.15	
batch.cs.rutgers.edu	0 %	0	0.40	0.18	
cpp.cs.rutgers.edu	0 %	0	0.28	0.20	
java.cs.rutgers.edu	0 %	1	0.53	0.30	
lisp.cs.rutgers.edu	0 %	0	0.38	0.12	
pascal.cs.rutgers.edu	0 %	0	0.48	0.15	
perl.cs.rutgers.edu	0 %	0	0.33	0.10	
prolog.cs.rutgers.edu	0 %	0	0.38	0.20	
python.cs.rutgers.edu	0 %	1	0.58	0.15	

▶ Specifications



H254 Machines Status (10 units)

Hostname	GPULoad	Logins	User_Cpu	System_Cpu	Misc
ice.cs.rutgers.edu	0 %	0	0.58	0.21	
snow.cs.rutgers.edu	0 %	0	0.58	0.13	
butter.cs.rutgers.edu	0 %	0	0.67	0.21	
cheese.cs.rutgers.edu	0 %	0	0.42	0.13	
candle.cs.rutgers.edu	0 %	0	0.42	0.08	
frost.cs.rutgers.edu	0 %	0	0.46	0.12	
popsicle.cs.rutgers.edu	0 %	0	0.42	0.17	
plastic.cs.rutgers.edu	0 %	0	0.50	0.12	
crayon.cs.rutgers.edu	0 %	0	0.42	0.17	
wax.cs.rutgers.edu	0 %	0	0.46	0.17	

How to use iLab

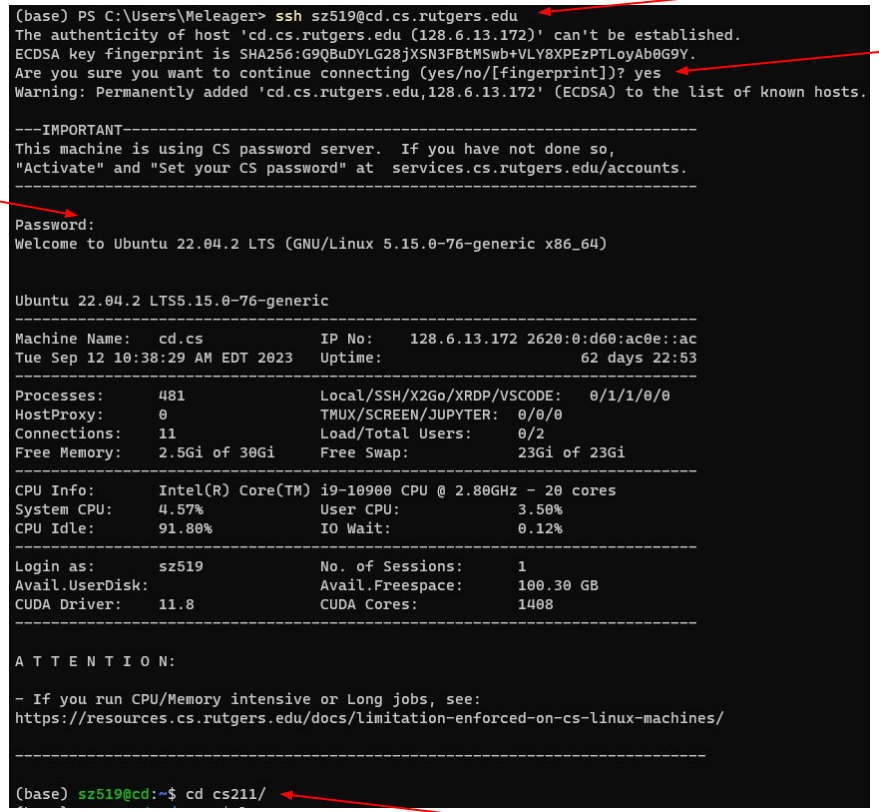
- SSH: Secure Shell or Secure Socket Shell
  - a network protocol/ secure way to access a computer over an unsecured network.
  - What you need:
    - IP address/ host name: machine_name.cs.rutgers.edu
    - Your username: your_net_id
    - Password: your_ilab_password
    - Port: 22 (optional, use on some ssh clients)
- Login command on Terminals:
  - ssh your_net_id@machine_name.cs.rutgers.edu
  - E.g. ssh hka22@basic.cs.rutgers.edu

How to use iLab

user_name@host_name

Say yes (only for the first time)

Password is invisible here when you type your password.



Use it as your local linux machine

File Transfer between your laptop and ilab

- scp: transfer files from local to server machine and vice versa
  - Local ➜ iLab: scp local-filepath netid@ilab.cs.rutgers.edu:~/remote-folder/
  - iLab ➜ local: scp netid@ilab. cs.rutgers.edu:~/remote-filepath local-filepath/
- Copy an entire folder: use “scp -r”
  - scp -r local-folder/ netid@ilab.cs.rutgers.edu:~/remote-folder/
  - scp -r netid@ilab.cs.rutgers.edu:~/remote-folder/ local-folder/

Linux commands

- pwd: show current directory/path
- ls: list all contents
- mkdir: create folder
  - mkdir cs336
- cd: change directory
  - cd cs336, or cd ..
- touch: create a file
  - touch test.txt
- nano: editor
  - nano test.txt
- grep: search texts in files
  - grep ‘cs211’ test.txt

Linux commands

- cat: print contents on screen or concatenate files
  - cat test.txt
  - cat test.txt test2.txt > test3.txt
- cp: copy file
  - cp source.txt destination.txt
- rm: remove file or folder
  - rm test.txt
  - rm -r myfolder
- mv: rename file or move file
  - mv oldname.txt newname.txt
  - mv test.txt another_folder

- **Introduction**
- **Accessing a DB**
- **SQL Examples**

Access PostgreSQL on iLab

```
(base) zc366@ilab1:~/Documents/postgresql-16.4$ psql --version
psql (PostgreSQL) 16.4
(base) zc366@ilab1:~/Documents/postgresql-16.4$ psql -l
                                                List of databases
   Name    | Owner | Encoding | Locale Provider |   Collate   |    Ctype    | ICU Locale | ICU Rules | Access privileges
-----------+-------+----------+-----------------+-------------+-------------+------------+-----------+-------------------
 postgres  | zc366 | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           |
 template0 | zc366 | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           | =c/zc366          +
           |       |          |                 |             |             |            |           | zc366=CTc/zc366
 template1 | zc366 | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           | =c/zc366          +
           |       |          |                 |             |             |            |           | zc366=CTc/zc366
 test      | zc366 | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           |
(4 rows)
```

Create a DB

```bash
createdb mydb # create a database named mydb
dropdb mydb # drop the database named mydb

psql -l # list all databases
# Don't forget to add .\ if Windows can't find the PATH.
```

```bash
psql mydb # connect to the database named mydb

mydb=# SELECT version(); # check the version of the database
mydb=# SELECT current_date; # check the current date
mydb=# SELECT 2 + 2; # check the result of the calculation

mydb=# \r # reset the query buffer(cancel the current query)
mydb=# \h # list all the commands
mydb=# \q # quit the database back to the terminal
```

Create Tables & Insert Values

```bash
psql -s mydb # single step mode to learn the SQL commands

mydb=# \dt # list all tables
DROP TABLE tablename; # drop the table named tablename
```

- TABLE weather
- TABLE cities
- Insert some data

```SQL
mydb=# \dt -- list all tables
mydb=# \d tablename # describe the table named tablename
mydb=# INSERT INTO tablename VALUES (xxx) -- insert a new record into the table
```

Querying a Table

- Select All
- Select with a condition
- Select with multiple conditions
- Select with sorting
- Select with distinct
- Select with limit ...

```SQL
SELECT * FROM weather; -- SELECT city, temp_lo, temp_hi, prcp, date FROM weather;
SELECT * FROM weather
    WHERE city = 'San Francisco' AND prcp > 0.0; -- AND, OR, NOT can be used in the WHERE clause
SELECT * FROM weather
    ORDER BY city, temp_lo; -- ORDER BY can be used to sort the result
SELECT DISTINCT city
    FROM weather; -- DISTINCT can be used to remove the duplicate rows
SELECT * FROM weather
    LIMIT 5; -- LIMIT can be used to limit the number of rows returned
```

Execute SQL Script

In the PG terminal, use \i filename to execute the file.

- Write SQL script (left figure)
- Execute in PG (right figure)

```
(base) jl3288@dbgroup:/data/local/jl3288/cs336$ cat test.sql
DROP TABLE IF EXISTS student;

CREATE TABLE student (
    id   INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);

INSERT INTO student (id, name) VALUES
    (1, 'Alice'),
    (2, 'Bob'),
    (3, 'Charlie'),
    (4, 'David');

SELECT * FROM student;

SELECT id, name
FROM student
WHERE id >= 2
ORDER BY name;
```

```
jl3288=# \i test.sql
psql:test.sql:1: NOTICE:  table "student" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 4
 id |  name
----+---------
  1 | Alice
  2 | Bob
  3 | Charlie
  4 | David
(4 rows)

 id |  name
----+---------
  2 | Bob
  3 | Charlie
  4 | David
(3 rows)
```

COPY command

- Importing data from csv file
  - \copy my_table FROM '/path/to/local/file.csv' WITH (FORMAT CSV, HEADER);
- Exporting data to csv file
  - \copy my_table TO '/path/to/local/file.csv' WITH (FORMAT CSV, HEADER);
- Importing using a specific delimiter
  - \copy my_table FROM 'data.txt' WITH (FORMAT TEXT, DELIMITER E'\t');
- Exporting specific columns
  - \copy my_table (column1, column2) TO 'output.csv' WITH (FORMAT CSV, HEADER);
- Exporting the result of a query
  - \copy (SELECT id, name FROM my_table WHERE active = true) TO 'active_users.csv' WITH (FORMAT CSV, HEADER);

Joins Between Tables

- **INNER JOIN**:
  Return matching rows between two tables, records that don't have matching rows are not included.
- **LEFT OUTER JOIN**:
  Return **all rows from the left table**, and **the matched rows from the right table**, the result is **NULL** from the right side if there is no match.
- **RIGHT OUTER JOIN**:
  Return **all rows from the right table**, and **the matched rows from the left table**, the result is **NULL** from the left side if there is no match.
- **FULL OUTER JOIN**:
  Return **all rows when there is a match in either left or right table**, the result is **NULL** from both sides if there is no match.

Joins Between Tables

- INNER JOIN and OUTER JOIN

```SQL
SELECT * FROM weather JOIN cities ON city = name; -- INNER JOIN, which is the default join
-- Try to find the city name in the weather table and the cities table
SELECT *
    FROM weather LEFT OUTER JOIN cities ON weather.city = cities.name; -- LEFT OUTER JOIN
-- Try to find the city name exists in the weather table and also in the cities table,
-- if not, fill the columns in the cities table with NULL
SELECT *
    FROM weather RIGHT OUTER JOIN cities ON weather.city = cities.name; -- RIGHT OUTER JOIN
SELECT *
    FROM weather FULL OUTER JOIN cities ON weather.city = cities.name; -- FULL OUTER JOIN
```

Aggregate Functions

- Aggregate functions
- GROUP BY
- HAVING

Aggregate functions perform a calculation on a set of values and return a single value.

```
COUNT() -- count the number of rows
SUM() -- sum the values
AVG() -- calculate the average
MAX() -- find the maximum value
MIN() -- find the minimum value
```

Aggregate Functions

- Aggregate functions
- GROUP BY
- HAVING

`GROUP BY` is a clause in SQL that is only used with aggregate functions. It group rows with the same values into summary rows, break the dataset into smaller groups based on the values of a specific column or columns, and then apply aggregate functions to each group.

```
SELECT product, SUM(sales)
FROM sales_table
GROUP BY product;
```

Aggregate Functions

- Aggregate functions
- GROUP BY
- HAVING

`HAVING` is used in combination with the `GROUP BY` clause to restrict the groups of returned rows to only those whose the condition is TRUE.

```
SELECT product, SUM(sales)
FROM sales_table
GROUP BY product
HAVING SUM(sales) > 1000;
```

Different from WHERE and HAVING

- Difference between `WHERE` and `HAVING`

`WHERE` is used **before** the data is grouped or aggregated, `HAVING` is used **after** the data is grouped or aggregated.

Because of this, `WHERE` can not be used with aggregate functions, but `HAVING` can.

- `WHERE` is used to filter records before any groupings are made, it can filter the **rows** with specific conditions(not related to aggregate functions).
- `HAVING` is used to filter values after they have been grouped, it can filter the **groups** with specific conditions(mostly aggregate functions).

Updates and Deletions

```SQL
UPDATE weather
    SET temp_hi = temp_hi - 2,  temp_lo = temp_lo - 2
    WHERE date > '1994-11-28';
-- Change the records in the weather table, set a condition to change the records
DELETE FROM weather WHERE city = 'Hayward';
```

Thank you!

Zihan